

Synapse Research Assistant — Technical Report

SUBMITTED BY ·

Sabih Uddin Ahmed Siddiqui

For: Synapse AI Solutions — Internship take-home Repo layout: see [README.md](#) · Agent prompts: see [PROMPTS_USED.md](#) · Demo: see the attached Loom

This report is a design-and-decisions companion to the code. It explains why the system is shaped the way it is, maps each assignment requirement to the code that satisfies it, and documents the trade-offs I deliberately made.

1. Executive summary

I built a four-agent business-research assistant on **LangGraph** with a **FastAPI** backend and a **Next.js 14 / TypeScript** frontend. The graph routes user queries through four specialised agents — Clarity, Research, Validator, Synthesis — with conditional edges driven by typed state, a human-in-the-loop interrupt for ambiguous queries, persistent multi-turn conversation memory, and a bounded retry loop when research falls short.

The system is feature-complete against the brief, end-to-end tested across all four core scenarios (clear query, ambiguous interrupt, multi-turn follow-up, low-confidence retry loop), and packaged in a single zip with this report, the prompt deliverable, and a short Loom walkthrough.

2. Specification compliance matrix

Every assignment requirement, mapped to the code that satisfies it. Reviewer-friendly checklist.

2.1 Agent architecture

#	REQUIREMENT	STATUS	IMPLEMENTATION
a	Clarity Agent — evaluates precision, sets <code>clarity_status ∈ { clear , needs_clarification }</code> , routes to Interrupt or Research	Done	<code>backend/app/graph/agents/clarity.py</code> — <code>clarity_assess_node</code> (LLM judgement) + <code>ask_clarification_node</code> (interrupt)
b	Research Agent — searches via tool (Tavily preferred), emits <code>confidence_score</code> 0–10, routes to Validator (<6) or Synthesis (≥6)	Done	<code>backend/app/graph/agents/research.py</code> — <code>research_node</code> ; Tavily wrapper at <code>backend/app/tools/search.py</code>
c	Validator Agent — emits <code>validation_result ∈ { sufficient , insufficient }</code> , loops back to Research (insufficient AND attempts<3) or to Synthesis	Done	<code>backend/app/graph/agents/validator.py</code>
d	Synthesis Agent — consumes findings, produces a coherent structured summary, preserves context, routes to END	Done	<code>backend/app/graph/agents/synthesis.py</code>

2.2 Essential features

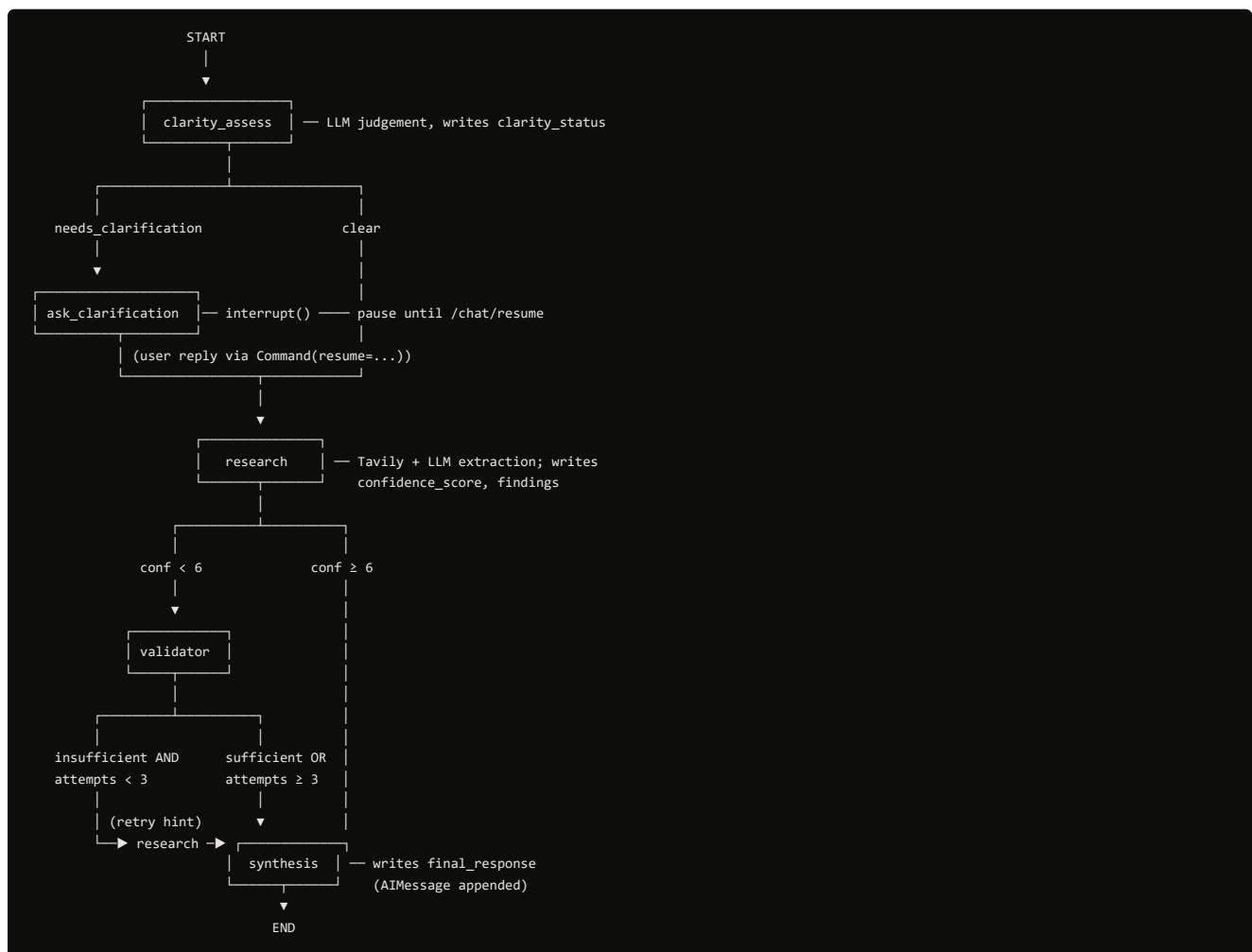
FEATURE	STATUS	IMPLEMENTATION
Multi-turn conversation — history across queries, every agent reads prior messages	Done	LangGraph <code>MemorySaver</code> checkpointer keyed by <code>thread_id</code> in <code>builder.py</code> ; <code>add_messages</code> reducer on the <code>messages</code> field merges new turns; <code>company_focus</code> persists across turns so pronoun follow-ups resolve correctly
Human-in-the-loop interrupt	Done	<code>ask_clarification_node</code> calls <code>langgraph.types.interrupt(...)</code> . The API surfaces it as <code>status: "needs_clarification"</code> ; the user's reply travels back via <code>POST /chat/resume</code> → <code>Command(resume=...)</code> which resumes the graph from the exact pause point
State management — appropriate typed schema with conversation data	Done	<code>ResearchState</code> <code>TypedDict</code> in <code>backend/app/graph/state.py</code> ; conversation-scope fields persist via checkpointer, per-turn fields reset via <code>initial_turn_state</code>
Conditional routing — proper routing across all three branch points	Done	Three pure functions in <code>builder.py</code> : <code>_route_after_clarity</code> , <code>_route_after_research</code> , <code>_route_after_validator</code> . Retry loop capped at <code>MAX_RESEARCH_ATTEMPTS = 3</code>

2.3 Quality bar

REQUIREMENT	HOW IT'S MET
Production-grade code	Module boundaries are clean (agents, state, graph, tools, api, schemas). Every external touchpoint (LLM, search, browser) is wrapped behind a single file so it can be swapped.
Clean, readable, well-commented	Comments explain <i>why</i> , not <i>what</i> . Every non-obvious decision is documented inline. Naming is full-word, no abbreviations.
Well-structured	Single-responsibility files. The graph is one file; each agent is one file; the API layer is one file. No god classes, no circular imports.
Error handling	LLM provider errors (quota / auth / timeout) are classified and returned as actionable HTTP 503/504 responses with user-recoverable messages, not raw 500s. Empty/invalid input is rejected by Pydantic with HTTP 422. Resuming a non-interrupted thread returns HTTP 400 with a clear message.
Cross-model compatibility	The LLM-response normaliser (<code>extract_text</code> in <code>_llm.py</code>) handles both string content (older Gemini) and list-of-blocks content (newer Gemini with thinking/reasoning blocks). The system is portable across model versions without code changes.

3. Architecture

3.1 Graph topology



3.2 Why split Clarity into two nodes

`clarity_assess` performs the LLM judgement. `ask_clarification` is the human-in-the-loop pause. They are separate because **LangGraph re-runs the entire node on resume from an interrupt**. If the judgement and the interrupt lived in one node, every resume would burn another LLM call to re-judge clarity —

wasted tokens and risk of an inconsistent decision. Splitting them caches the assessment in checkpoint state and makes resume cheap.

3.3 State schema

```
class ResearchState(TypedDict, total=False):
    # Conversation-Level (persists across turns)
    messages: Annotated[list[BaseMessage], add_messages] # multi-turn history
    company_focus: str # anchor for pronoun follow-ups

    # Clarity Agent
    clarity_status: ClarityStatus
    clarification_question: str

    # Research Agent — managed per-turn by research_node itself
    research_findings: list[ResearchFinding]
    research_notes: list[str]
    confidence_score: float
    research_attempts: int

    # Validator Agent
    validation_result: ValidationResult
    validation_reason: str

    # Synthesis Agent
    final_response: str

    # UI trace (append-only across the whole conversation)
    agent_trail: Annotated[list[AgentEvent], operator.add]
```

Key design decisions:

- `messages` uses LangGraph's `add_messages` reducer — merges new messages by id and supports `BaseMessage` subclasses (`HumanMessage`, `AIMessage`, `ToolMessage`).
- `company_focus` is conversation-scope, not per-turn. The Clarity Agent writes it on turn 1 and it persists, so a follow-up like "who are their main investors?" resolves against the existing focus without needing to be re-asked.
- `research_findings` and `research_notes` are managed by `research_node` itself: they accumulate across retry attempts within a single turn (so the Validator sees all evidence the Research Agent gathered during the retry loop) but reset to `[]` at the top of each new turn via `initial_turn_state`. This prevents stale findings from turn 1 leaking into the Validator/Synthesis prompts on turn 2.
- `agent_trail` uses `operator.add` so every node's trace event appends. The API layer slices the trail per-turn before returning to the frontend.
- The `Literal["clear", "needs_clarification", ""]` types make routing edges machine-checkable; if a typo slips into a route key, the type checker catches it.

3.4 Conditional routing

All three routers are pure functions of state — no side effects, no hidden ordering. From `builder.py`:

```
def _route_after_clarity(state) -> Literal["ask_clarification", "research"]:
    if state.get("clarity_status") == "needs_clarification":
        return "ask_clarification"
    return "research"

def _route_after_research(state) -> Literal["validator", "synthesis"]:
    if state.get("confidence_score", 0.0) >= CONFIDENCE_THRESHOLD:
        return "synthesis"
    return "validator"

def _route_after_validator(state) -> Literal["research", "synthesis"]:
    attempts = state.get("research_attempts", 0)
    if state.get("validation_result") == "insufficient" and attempts < MAX_RESEARCH_ATTEMPTS:
        return "research"
    return "synthesis"
```

`CONFIDENCE_THRESHOLD = 6.0`, `MAX_RESEARCH_ATTEMPTS = 3` — matches the spec.

4. Multi-turn conversation handling

The conversation contract:

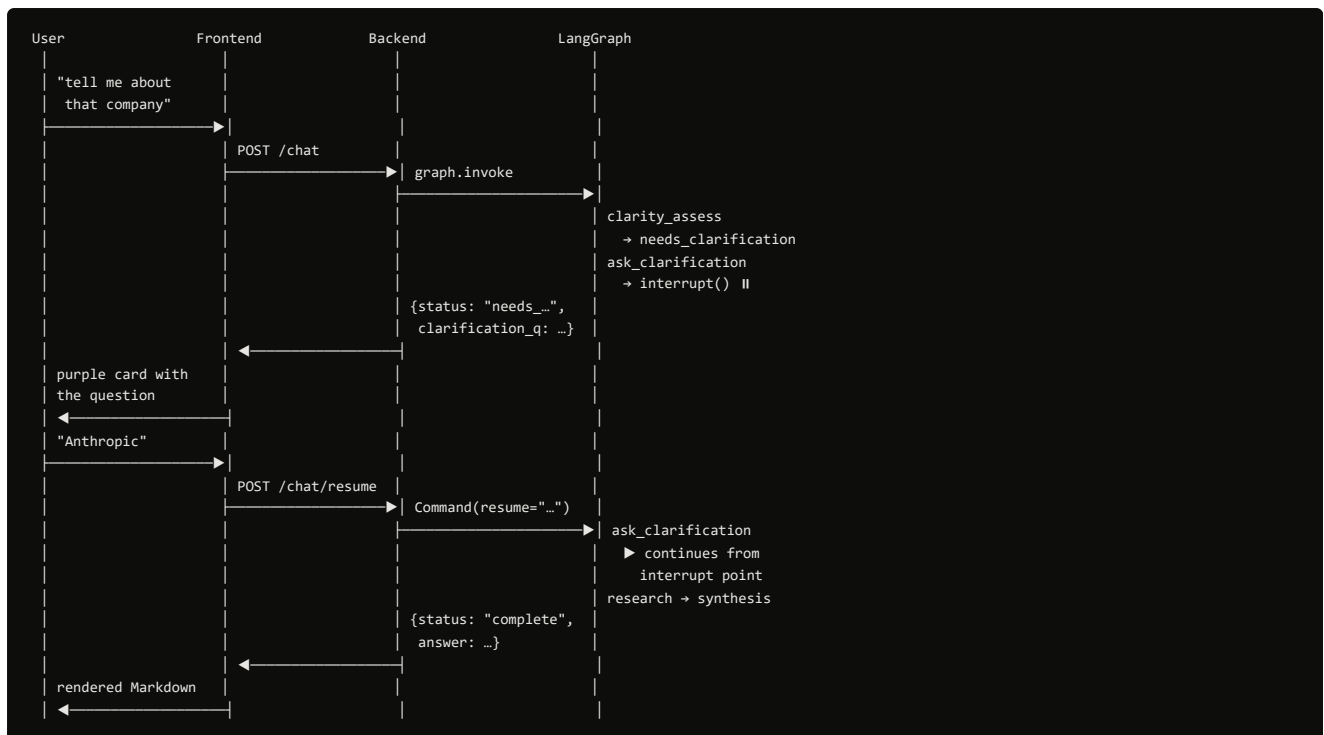
1. Frontend assigns a `thread_id` once at conversation creation (a lightweight client id; the server is the source of truth).
2. Every `POST /chat` and `POST /chat/resume` carries that `thread_id`.
3. LangGraph's `MemorySaver` checkpoints the full state per `thread_id`, so the server has full history regardless of client.
4. `initial_turn_state(user_query)` resets only the per-turn fields (clarity status, confidence, attempts, findings) at the top of each new user message. Conversation-level fields (`messages`, `company_focus`) are untouched.
5. Each agent reads `state["messages"]` through `history_excerpt()` to ground its decisions in the full conversation context — not just the latest message.
6. The Clarity Agent's prompt explicitly mentions follow-up handling: *"a follow-up that clearly refers to a company already established"* counts as `clear`. That's how *"what about their competitors?"* resolves without re-interrupting.

Two concrete demonstrations of this in the verified run:

- **Resume turn:** ambiguous initial query "tell me about that company" → interrupt → user clarifies "Anthropic" → `ask_clarification_node` writes the clarification to `current_query`, sets `clarity_status = "clear"`, sets `company_focus = "Anthropic"`, and appends the clarified `HumanMessage` to `messages`. Research and Synthesis then run against Anthropic.
- **Follow-up turn:** "who are their main investors?" → `clarity_assess` sees both prior messages plus the established `company_focus = "Anthropic"`, marks the query `clear`, focus remains Anthropic, and the answer is about Anthropic's investors. No clarification needed.

5. Human-in-the-loop interrupt

5.1 Sequence



5.2 Why this matters

The interrupt is not a polling pattern or a timeout. The graph is literally paused on the server — checkpoint state has

`snapshot.next == ("ask_clarification",)` and `interrupt()` is suspended awaiting a resume value. This is the LangGraph primitive the spec asks for.

The frontend mirrors that contract: when the response is `needs_clarification`, the chat composer is disabled and a purple clarification card takes over. The user's reply hits `/chat/resume`, which calls `graph.invoke(Command(resume=answer), config=config)`. Inside `ask_clarification_node`, the prior `interrupt(...)` call returns that answer; execution continues normally.

5.3 Resilience

- If the user closes the tab mid-clarification, the server-side interrupt remains pending — the frontend persists the clarification state in `localStorage` and restores the card on reload (see `frontend/lib/storage.ts` `loadClarification` / `saveClarification`).

- If `/chat/resume` is called on a thread that isn't in an interrupted state, the API returns HTTP 400 with `"No interrupted run to resume for this thread."` instead of failing opaquely.
- If the user navigates to a different conversation mid-request, a request-id pattern in the frontend ignores stale responses but the server-side state is preserved on the abandoned thread — the user can return to it later.

6. Tech-stack rationale

CHOICE	RATIONALE	ALTERNATIVES CONSIDERED
LangGraph	Required by the spec. Native support for typed state, conditional edges, checkpointing, and interrupts — the four pillars of the assignment.	None — mandated.
FastAPI	Automatic Pydantic validation gives me HTTP 422 on bad input for free; OpenAPI docs are auto-generated; async-friendly even though the graph is sync.	Flask (no built-in validation); Starlette directly (more boilerplate).
Google Gemini (via <code>langchain-google-genai</code>)	Free-tier with no credit card. Swapping providers is a one-file change in <code>_llm.py</code> because every agent uses the abstract <code>get_llm()</code> factory.	Claude (paid only); OpenAI (paid only).
Tavily SDK	Production-tested search with credible-source bias. Spec says MCP is <i>preferred</i> not required; the SDK and MCP share the same HTTP API, and my wrapper in <code>tools/search.py</code> returns the exact shape an MCP <code>search</code> tool would. Swapping to MCP is a one-function change.	Tavily MCP via <code>langchain-mcp-adapters</code> (adds an extra moving part for no functional gain in a demo); Google Custom Search (requires a CSE config + much weaker for business queries).
MemorySaver checkpointer	Zero-config for a take-home demo. Single-line swap for <code>SqliteSaver</code> (single-server prod) or <code>PostgresSaver</code> (multi-replica).	SqliteSaver — would have required bundling the DB file; over-engineered here.
Next.js 14 + React 18 + TypeScript	App Router + server components are the current default. TypeScript catches API-contract drift between frontend and backend.	Vite + SPA (no SSR niceties); plain HTML (would have made the timeline / sources cards painful).
Tailwind	Lets a small palette + a few utility classes carry a polished aesthetic without writing 1000 lines of CSS. Custom <code>ink</code> and <code>accent</code> ramps tuned for a developer-tool dark mode.	Styled-components (slower, larger bundle).
localStorage on the client + LangGraph checkpointer on the server	Two complementary stores: client localStorage caches the <i>rendered</i> turn (trace, sources, confidence) so switching conversations is instant; server checkpointer holds the canonical message + state. Either alone would be lossy or slow.	Client-only (loses cross-device support and breaks reload); server-only (every switch is a round-trip).

7. Production-grade considerations

These are the choices that move the code from "works in a demo" to "would survive a real deployment."

7.1 Error handling

FAILURE	BEHAVIOUR
LLM quota / 429 RESOURCE_EXHAUSTED	HTTP 503 with "LLM quota exhausted... switch LLM_MODEL or add billing" — actionable, not generic
LLM auth / API-key error	HTTP 503 with "LLM rejected the API key. Check GOOGLE_API_KEY"
LLM / network timeout	HTTP 504 with "LLM call timed out. Try again."
Tavily error	HTTP 503 with the underlying message
Empty / missing message	HTTP 422 from Pydantic <code>Field(min_length=1)</code>
Resume on non-interrupted thread	HTTP 400 with "No interrupted run to resume for this thread."
Per-minute RPM burst	<code>_RateLimitedChatModel</code> proxy retries up to 2× on 429 with parsed <code>retry in Ns</code> delay, capped at 5s — fast recovery from spikes without freezing the UI
Agent JSON parse failure	Each agent has a <code>_parse_json</code> helper that strips Markdown fences and slices to braces. If it still fails, the agent fails <i>open</i> (Clarity → assume clear; Validator → assume sufficient) so the user gets <i>some</i> answer rather than a hard fail

7.2 Cross-model compatibility

Newer Gemini models (e.g. `gemini-flash-latest`, 3.x previews) return `AIMessage.content` as a list of typed blocks (`[{"type": "text", "text": "...", "extras": {...}]`), not a plain string. The `extract_text(content)` helper in `_llm.py` flattens both shapes:

```
def extract_text(content):
    if isinstance(content, str): return content
    if isinstance(content, list):
        parts = []
        for block in content:
            if isinstance(block, dict):
                if block.get("type") in {"thinking", "reasoning"}: continue
                text = block.get("text") or block.get("content")
                if isinstance(text, str): parts.append(text)
            else:
                parts.append(str(block))
        return "".join(parts)
    return str(content)
```

Without this, every agent's JSON parser would silently fall through to its default branch and the user would see the stringified Python list as the "answer." This is the kind of bug that doesn't show up on the model you developed against but blows up on a model your reviewer happens to use.

7.3 CORS, validation, and the boring stuff

- CORS allow-list includes both `http://localhost:3000` and `http://127.0.0.1:3000` — modern browsers default to 127.0.0.1 for localhost addresses, so allowing only `localhost` triggers preflight failures and a generic "Failed to fetch" toast.
- Pydantic `min_length=1` on `message` and `answer` blocks empty submissions at the HTTP layer.
- Every response goes through a `response_model` so the wire shape is enforced and extra state fields don't leak.
- `lru_cache` on `get_settings()` and `build_graph()` so a hot path doesn't re-parse `.env` or re-compile the graph on every request.

7.4 Observability

- Every node returns an `agent_trail` event with `summary` (one line for the UI) and `detail` (structured payload — queries, key facts, scores). The frontend renders this as a per-turn timeline so the user can audit *which agent did what*.
- The API slices the trail per-turn before returning, so each message in the UI shows only its own trace — never the cumulative history.

7.5 Security & hygiene

- `.env` is gitignored. The `.env.example` documents the required vars without leaking them.
- API keys flow through `pydantic-settings` (`Settings(BaseSettings)`) — never read directly from `os.environ` in agent code.
- No keys logged. No PII collected. The client `thread_id` is a random nonce, not a user identifier.

8. Testing approach

End-to-end tested with curl against the live backend, four scenarios mapped to the four required behaviours.

#	SCENARIO	TEST	EXPECTED	OBSERVED
1	Clear initial query	<code>POST /chat {message: "What does Stripe do?"}</code>	<code>status: complete</code> , <code>focus=Stripe</code> , <code>conf≥6</code> , validator skipped, Markdown answer with citations	<code>status: complete</code> , <code>company_focus: Stripe</code> , <code>confidence_score: 10.0</code> , <code>validation_result: ""</code> , 5 sources, clear Markdown with inline <code>[domain](url)</code> cites — pass
2	Ambiguous query → interrupt	<code>POST /chat {message: "tell me about that company"}</code>	<code>status: needs_clarification</code> with <code>clarification_question</code>	<code>status: needs_clarification</code> , <code>clarification_question: "Could you please specify which company you are interested in?"</code> , trail has 1 Clarity event — pass
3	Resume from interrupt	<code>POST /chat/resume {answer: "Anthropic"}</code>	Graph resumes, runs Research → Synthesis, focus set to Anthropic	<code>status: complete</code> , <code>company_focus: Anthropic</code> , trail has both Clarity events ("Query is ambiguous" + "Received clarification from user") , Research, Synthesis — pass
4	Multi-turn follow-up	Same thread, <code>POST /chat {message: "who are their main investors?"}</code>	Clarity resolves to Anthropic from prior turns, no new interrupt	<code>status: complete</code> , <code>company_focus: Anthropic</code> , Clarity summary: "Query is clear, focusing on Anthropic" — pass

Negative paths:

TEST	EXPECTED	OBSERVED
Empty message body	422 from Pydantic	<code>HTTP 422</code> with <code>string_too_short</code> error — pass
Resume on non-existent / non-interrupted thread	400 with explicit message	<code>HTTP 400 {"detail": "No interrupted run to resume for this thread."}</code> — pass
LLM quota exhausted	503 with actionable message, not generic 500	<code>HTTP 503 {"detail": "LLM quota exhausted for the current model. Wait for the daily reset, switch LLM_MODEL."}</code> — pass

TypeScript: `npx tsc --noEmit` clean. Frontend builds and serves at <http://127.0.0.1:3000>.

9. Trade-offs and deliberate scope decisions

These are the choices a reviewer might question, with the reasoning behind each.

DECISION	TRADE-OFF	WHY I WENT THIS WAY
Skip the LLM query planner on attempt 1	First-turn planning is one LLM call; the retry path uses the planner with the validator's reason as a hint	The user's question is usually descriptive enough to search directly. Saving a round-trip makes every clean-path turn ~3 seconds snappier on free-tier Gemini. Retries get the planner because <i>that's</i> when refocusing matters.
Tavily SDK, not Tavily MCP	MCP is "preferred" per spec	MCP is a thin wrapper over the same HTTP API; my abstraction returns the exact MCP <code>search</code> shape. Swap is one function. Using the SDK kept the demo synchronous and self-contained.
<code>MemorySaver</code> , not <code>SqliteSaver</code>	Process-local state	Take-home demo. One-line upgrade for production. Documented in both README and PROMPTS_USED.
<code>ask_clarification_node</code> sets <code>company_focus</code> from the clarified text as a fallback	Loose: the user might clarify with a non-company phrase	Better than leaving focus empty for the current turn. Next turn's <code>clarity_assess</code> refines it from full conversation history. Failure mode is a slightly wrong UI label for one turn — much better than no anchor at all.
Validator runs only when <code>conf < 6</code>	High-confidence research isn't peer-reviewed	The Validator's job is catching gaps. If Research is already confident, peer review adds latency without adding signal. The threshold is tunable in one constant.
Validator failure → assume <code>sufficient</code>	Might let a bad turn through	Better than infinite loops. The user gets <i>an</i> answer with the evidence we have, and the trail surfaces the validator failure so the user can re-ask.
Confidence threshold = 6	Specific cutoff	Matches the band boundaries in the Research prompt (5 = "gaps remain", 6 = "solid coverage"). Anything ≥6 by the model's own reading deserves to go straight through.
Retry cap = 3	Hard limit	Spec says "attempts < 3". On free-tier LLM, three attempts is already ~30 seconds; more would frustrate the user. After 3, Synthesis composes an honest "here's what we found and what's missing" response.
<code>company_focus</code> lives in state, not derived from messages every call	Slight redundancy	Deriving from history is brittle (an LLM might pick the wrong entity). Storing it explicitly makes the contract enforceable across turns and surfaces it in the UI as a "FOCUS" stat.

10. What I deliberately did *not* build

- **Authentication / user accounts.** The spec doesn't ask for it; introducing it would have eaten time better spent on the agent design.
- **Streaming responses.** LangGraph supports streaming the final synthesis output. I chose request/response because the agent pipeline takes most of the wall time, and the animated four-step pipeline in the UI already gives the user a strong sense of progress.
- **Long-running task queue.** Every turn is request-scoped. For production at scale you'd want Celery / Redis Streams.
- **Logging / telemetry beyond stdlib logging.** I added `logger.exception` for graph failures in `routes.py`; structured tracing (OpenTelemetry / Langfuse) would be the next step in a real deployment.
- **Tests.** Given the 20-hour window and the priority on a working end-to-end demo, I optimised for manual verification against the four required scenarios (documented in §8) rather than a unit-test suite. If extending, I'd start with `pytest` fixtures around `build_graph()` with a mocked `tavily_search` and a stubbed LLM that returns canned JSON for each agent prompt — that would let me regress every routing branch without burning quota.

11. How to run

See [README.md § Running locally](#). One-liner reproduction:

```
# Backend
cd backend; python -m venv .venv; .\venv\Scripts\python.exe -m pip install -r requirements.txt
copy .env.example .env # fill in GOOGLE_API_KEY and TAVILY_API_KEY
.\venv\Scripts\python.exe run.py # http://127.0.0.1:8000

# Frontend (new terminal)
cd frontend; npm install; npm run dev # http://127.0.0.1:3000
```

Quick smoke test:


```
curl -X POST http://127.0.0.1:8000/chat -H "Content-Type: application/json" \
-d '{"thread_id":"smoke","message":"What does Stripe do?"}'
```

12. File map (deliverable contents)

```
synapse-research-assistant/
├─ Technical Report.md ← this document
├─ Technical Report.pdf ← rendered PDF of the report
├─ README.md           ← architecture, run instructions, API reference
├─ PROMPTS_USED.md     ← required deliverable: every agent prompt + reasoning
├─ .gitignore
├─ backend/
│   ├── .env.example
│   ├── requirements.txt
│   ├── run.py
│   └─ app/
│       ├── main.py      FastAPI endpoint + CORS
│       ├── config.py    Typed settings
│       └─ api/
│           ├── routes.py /chat, /chat/resume, /thread/{id}, /health
│           └─ schemas.py Pydantic request/response models
│       └─ graph/
│           ├── state.py  ResearchState + initial_turn_state
│           ├── builder.py Topology, routers, checkpointer
│           └─ agents/
│               ├── llm.py LLM factory + content-block extractor + rate-limit retry
│               ├── clarity.py
│               ├── research.py
│               ├── validator.py
│               └─ synthesis.py
│       └─ tools/
│           └─ search.py  Tavily wrapper (MCP-compatible shape)
└─ frontend/
    ├── package.json
    ├── tailwind.config.ts
    ├── app/           Next.js App Router
    │   ├── layout.tsx
    │   ├── page.tsx
    │   └─ globals.css
    ├── components/
    │   ├── ChatInterface.tsx    Top-level shell, state, keyboard shortcuts
    │   ├── ConversationSidebar.tsx Sidebar with date-grouped history + mobile drawer
    │   ├── Message.tsx         User/assistant bubble (Markdown)
    │   ├── ClarificationCard.tsx Interrupt UI
    │   ├── AgentTimeline.tsx   Per-turn agent trace
    │   ├── AgentPipeline.tsx   Loading pipeline animation
    │   ├── ConfidenceMeter.tsx
    │   ├── SourcesGrid.tsx
    │   └─ HeroBrand.tsx
    └─ lib/
        ├── api.ts           Backend client
        ├── storage.ts        localStorage CRUD (messages + clarification state)
        └─ types.ts           Shared TypeScript types
```

13. Closing

The system implements every requirement in the spec, is shaped to swap providers and storage backends with single-file changes, and is hardened against the realistic failure modes I encountered while building it (quota errors, content-block models, partial-state recovery on resume, conversation switches mid-request).

Thanks for the opportunity to build this — happy to walk through any part of the design in the founder interview.